

PlanMap

A VS Code extension that plans before it codes

Product Requirements Document — v0.2 (Draft, plain-language version)

Prepared by: Sambhav

Status: Pre-build / Concept validated, MVP scoping stage

What this is, in one sentence: PlanMap is a VS Code extension that turns a feature request into a small, visual plan you can check and approve before any code gets written — and it keeps watching afterward to make sure the code still matches what you approved.

How to read this document: This is written to be readable start to finish, in plain language, without needing outside context. Technical terms are explained the first time they appear, and Section 3 is a glossary you can jump back to any time a word is unclear.

1 The Problem, in Plain Terms

Tools like Cursor, GitHub Copilot, and Windsurf work like a chat conversation. You type a short instruction — "move this button to the right" — and the AI decides on its own what that means and how to change your code. Most of the time this works fine. But when it doesn't, it fails in a specific, repeatable way, and that failure is the whole reason this product exists.

Here is the real example that started this idea:

A developer asked an AI agent to remove a function from their code. Instead of deleting it cleanly, the agent left the old function in place and quietly wrote a second function whose only job was to turn the first one off. The app still worked. Nothing crashed. But now there were two pieces of code doing the job of one, and nobody had decided that on purpose — the AI just picked the easiest path that made the immediate request "succeed."

This kind of thing keeps happening, for a few connected reasons:

- **The AI decides instead of just doing what you meant.** A short instruction gets reinterpreted broadly, and the AI can end up rewriting or overlapping code you never asked it to touch.
- **It patches instead of properly fixing things.** Asked to remove something, it may instead bury a workaround next to it. The code still runs, so it "looks" done — but the codebase quietly gets messier every time this happens.
- **It forgets everything between conversations.** Every new chat starts from zero. Decisions you already made, and reasons you made them, aren't remembered — so you either repeat yourself constantly, or the AI silently re-decides something that was already settled.
- **There's no "are you sure?" step.** The only thing you ever see is the finished change. There's no in-between moment where you can look at the plan before the code exists and say "yes, that's right" or "no, do it differently."
- **You slowly stop understanding your own project.** This hits newer developers hardest. If the AI is always the one deciding how things should be built, the human using it never actually learns or owns the reasoning behind the system they're supposedly building.

This exact pattern now has an industry name: **"vibe coding"** — accepting AI-written changes without really reviewing them or keeping a record of why they were made. It's become common enough that a whole new way of working, called **spec-driven development**, has emerged in the last year specifically to fix it (tools like AWS Kiro, GitHub Spec Kit, and OpenSpec all exist because of this exact problem). We'll explain what that term means, and how PlanMap fits into it, in the next two sections.

2 The Idea, in Plain Terms

PlanMap sits between you and the AI coding agent you already use (Claude Code, Copilot, Cursor — whichever one you have). It doesn't replace that agent or try to write better code than it does. Its job is simpler and narrower: **make sure a decision gets made and reviewed before the agent is allowed to act on it, and keep track of that decision afterward.**

Think of it like this everyday comparison:

Today, using Cursor or Copilot is like handing a contractor a one-line text message — "add a bathroom" — and letting them start knocking down walls immediately, with no blueprint, no discussion, and no record of what was agreed. It might turn out fine. It might also mean they moved a load-bearing wall because your one sentence didn't specify enough. PlanMap is the step where the contractor first shows you a simple sketch of what they're about to build, you say yes or change it, then they start work — and afterward, if something gets changed later that doesn't match the sketch, someone notices.

Three simple rules the whole product is built around:

- **Plan first, code second — but only when it's worth it.** For a real feature, PlanMap generates a short plan grounded in your actual code (not a generic guess) and you approve it before anything is written.
- **Small changes don't get slowed down.** A one-line typo fix or a CSS tweak skips the whole planning step and goes straight through — more on exactly how in Section 5.
- **The plan doesn't go stale.** Once code is written, PlanMap remembers which plan it came from. If that code is edited later in a way that breaks from the original plan, PlanMap notices and tells you — instead of you finding out three weeks later that nobody remembers why something was built a certain way.

3 Glossary — Key Terms Used in This Document

Term	What it means here
Spec / plan	A short written description of what should be built and why, created before any code is written.
Node	One single piece of the plan — for example "users can reset their password" is one node. A plan is made of several connected nodes.
Graph	The visual picture made of nodes connected to each other, shown in a panel inside VS Code — this is how you see and review the plan.
Drift	When the actual code no longer matches what its plan node said it should do — usually because someone changed the code later without updating the plan.
Blast radius	A simple way of asking "how much stuff does this change touch?" A typo fix has a tiny blast radius. Adding a payment feature has a large one.
Fast path	The shortcut PlanMap uses for small, safe changes — no plan, no review screen, straight to the fix.
Structured path	The full plan-then-approve-then-build process, used for anything bigger or riskier than a small fix.
Lens	A way of viewing the same plan graph from a different angle — for example a "security lens" highlights only the parts of the plan that touch authentication or user data.
Coding agent	The AI tool that actually writes the code — Claude Code, Copilot, or Cursor's agent. PlanMap directs it; it doesn't replace it.

4 Who Else Is Doing Something Like This

It's worth being upfront: this general space isn't empty. A few tools already solve parts of this problem well, and PlanMap shouldn't waste time rebuilding what they've already solved. Here's the honest picture:

Tool	What it already does well	What it doesn't do
DeepWiki (by Cognition, the Devin team)	Explains a repo that's <i>already finished</i> — diagrams and an AI chat you can ask questions to. Genuinely popular: around 900,000 visits a month, free for public repos.	It only looks backward at finished code. It doesn't help you plan new work, and it can't tell you when code has quietly drifted from what was intended.
AWS Kiro, GitHub Spec Kit, OpenSpec	Write a plan (requirements, design, tasks) before letting an AI touch any code — this is called "spec-driven development," and it's the closest thing to PlanMap that exists today.	The plan is a set of markdown text files, written once. Nothing automatically checks later whether the code still matches that plan — it can quietly go stale.
Augment Cosmos	Remembers decisions and context across a whole company's codebase over time.	Not built around reviewing one feature at a time, and has no visual plan you look at and approve.
Cursor, Copilot, Claude Code	Very good at actually writing code once told what to do. PlanMap sends approved plans to these tools rather than competing with them.	No planning step, no memory between sessions, no way to notice later drift.

So what's actually new about PlanMap? Three things, put together in a way nobody else currently does: the plan is a **picture you can look at** instead of files you have to read top to bottom; the plan and the code stay **connected after the fact**, so drift gets caught instead of ignored; and the tool **only asks for a full review when it's actually worth it**, instead of treating every change the same way.

5 A Walk-Through Example

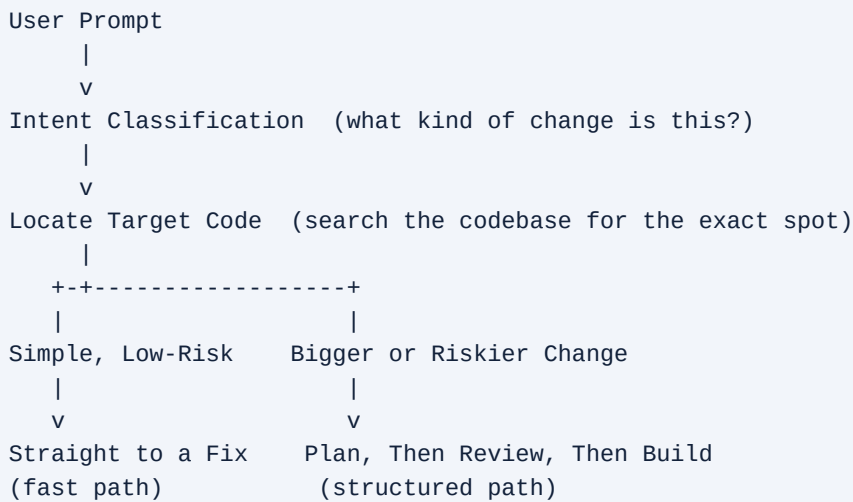
The easiest way to understand PlanMap is to follow one real request through the system, start to finish. Say a developer types: *"Add a password reset feature."*

- **1. PlanMap checks how big this request is.** "Add password reset" touches several things (email, auth, database) and isn't trivial to undo, so it's routed to the full review process instead of the quick fast path.
- **2. PlanMap looks at the real codebase first.** It finds the existing login code, the current user database table, and how emails are currently sent in this project — so the plan it's about to write is based on how this app actually works, not a generic guess.
- **3. A plan is generated as a small graph, not a paragraph.** One node says *"user can request a password reset"* (the business goal). A second node says *"send a time-limited reset link by email"* (the design decision). A third node says *"rate-limit attempts and hash the token"* (the security consideration). Underneath, a few small task nodes: "add reset endpoint," "add email template," "add rate limiting."
- **4. The developer looks at the graph before anything is built.** Maybe the rate-limit number is wrong, or the developer wants magic links instead of a token — they edit the node directly, right there, before a single line of code exists.
- **5. Once approved, each task node is sent, one at a time, to the coding agent already installed** — Claude Code, Copilot, whichever the developer uses. Each task only gets the small slice of code it actually needs, not the whole project, which is what keeps the AI from wandering off and "fixing" things nobody asked it to touch.
- **6. PlanMap remembers what got built and where.** The "rate-limit attempts" node is now linked to the exact function that implements it.
- **7. Weeks later, someone edits that rate-limiting code** — maybe removing the limit entirely during a rushed fix. PlanMap notices the code no longer matches what that node promised, and flags it as *drifted* right on the graph. instead of that change quietly slipping through unnoticed.

That last step is the part that doesn't exist anywhere else today — a plan that keeps checking itself against reality, instead of being written once and forgotten.

6 How a Request Is Handled, Step by Step

Every request goes through the same first checkpoint:



PlanMap decides which path to take by asking three simple questions about the request: **How much does this touch?** (a typo fix touches one line; a new feature touches many files) — **How easy is this to undo if it's wrong?** (a style tweak is harmless; anything touching passwords, payments, or user data is not) — and **Have we seen this before?** (is this changing something the plan already has a node for, or is it something genuinely new). If the answer is ever unclear, PlanMap doesn't guess — it plays it safe and sends the request through the full review process instead. Guessing wrong here quietly is exactly the failure PlanMap is trying to prevent.

6.1 The Fast Path — for small, low-risk changes

- **Figure out the request type.** A quick check reads the request and tags what kind of change it is — move something, rename something, remove something, fix a small bug.
- **Find the exact spot in the code.** Instead of handing the whole project to the AI and hoping it finds the right place, PlanMap searches the codebase directly and finds the specific file and lines involved.
- **Only proceed if it's clear.** If there's exactly one obvious match, PlanMap continues. If there are several possible matches — say, three different buttons that could be "the button" — it either asks a quick one-line question or sends the request to the full review process instead of guessing.
- **Hand over only what's needed.** The coding agent is only shown the specific piece of code it needs to change — not the whole file. This is what stops it from "fixing" the problem by quietly changing something else nearby.
- **Still write it down.** Even a tiny change gets a one-line note saved, so there's always a record of what changed and why — nothing disappears, even outside the full planning process.

6.2 The Structured Path — for real features

- **Look at the real code first.** Before writing any plan, PlanMap pulls in the actual related files, so the plan reflects how this specific project already works.

- **Generate the plan as a graph.** One node for the business goal, one for the design decision, one for security/backend considerations, and small task nodes underneath — this is what you saw in the walk-through example above.
- **The developer reviews it.** The graph opens in a panel inside VS Code. Every node can be approved, edited, merged with another, or thrown out. Nothing gets built until a node is approved.
- **Approved work goes to the coding agent, one piece at a time.** Each task node is handed off with only the context it needs — not the whole feature at once.
- **PlanMap keeps watching afterward.** Once a task is done, PlanMap remembers exactly which code it produced. From then on, if that code changes in a way that no longer matches what the node said it should do, the node is marked "drifted" so it's impossible to miss.

7 What Gets Saved, and Where

The plan graph isn't kept in a chat history that disappears — it's saved as ordinary files inside your project, in a folder called `.planmap/`. Because it's just files, it can be checked into git like any other part of your code, shared with teammates, and it's still there the next time you open the project — this is what solves the "every session starts from zero" problem directly.

Here's what one single node looks like when saved. You don't need to write this by hand — PlanMap generates and manages it — but it's useful to see what's actually being remembered:

```
{
  "id": "node_0143",
  "type": "task",                      // business | design | security | task
  "title": "Rate-limit reset attempts",
  "status": "implemented",            // pending | approved | implemented | drifted
  "parent": "node_0140",
  "description": "Max 5 reset requests per email per hour.",
  "linked_files": [
    { "path": "src/auth/reset.ts", "range": [42, 78], "hash": "a91f..." }
  ],
  "lens_tags": ["security", "backend"],
  "created_at": "2026-07-01T10:12:00Z",
  "last_verified": "2026-07-03T09:00:00Z"
}
```

In plain terms: this node knows what it's supposed to do, whether it's been built yet, exactly which lines of code it produced, and when it was last checked against that code. That last part — `last_verified` — is what makes drift detection possible.

8 Lenses — Different Views of the Same Plan

The plan graph doesn't change depending on who's looking at it — but how it's *grouped and highlighted* can. A lens is just a different way of looking at the exact same nodes.

Lens	Shows you	Useful for
Business	What the feature actually does for the user, step by step.	Anyone — including non-engineers
Backend	How a request flows through the system, service by service.	Backend developers
Security	Only the parts that touch logins, permissions, or user data.	Security-minded review

*To keep the first version small and shippable, only the **Business** and **Security** lenses are planned for v1. The Backend lens is a natural next step, not a launch requirement.*

9 What It's Built With

This section is a quick reference for how the pieces fit together technically:

- **The shell:** a normal VS Code extension — a sidebar for quick status, and a bigger panel (built with a graph library like React Flow) for the interactive plan graph itself.
- **The "thinking" behind the plan:** whatever AI access the developer already has — their Claude API key, their Copilot subscription. PlanMap doesn't run or train its own AI model.
- **Finding the right code:** a search over the project's files (by name, by symbol, and as a fallback, by meaning) — this only searches the current project, not the whole internet or a huge index rebuild every time.
- **Writing the code:** handed off entirely to whichever coding agent is already installed — Claude Code, Copilot, or Cursor. PlanMap organizes and checks the work; it doesn't try to write better code than these tools already do.
- **Choosing between AI models:** deliberately left out of v1. If this is ever needed later, there are already free, ready-made tools for it (like LiteLLM's router) — building this from scratch would be wasted effort on a problem that's already solved elsewhere.
- **Where everything is stored:** plain JSON files inside the project's own `.planmap/` folder, saved with the rest of the code. No separate server or database is needed for the first version.

10 Build Order — What Gets Built First

This is ordered so that each step is useful on its own, and the earliest, cheapest step tests the riskiest assumption before any real engineering time is spent.

Step	What gets built	What it tells us
0	No code yet — just sketch one real feature from Sentinel as a plan graph, by hand, in Figma or on paper.	Whether a graph is actually more useful than a text document, before writing a single line of the extension.
1	Show a plan graph inside VS Code, reading from a JSON file.	That the visual side of the tool works and feels good to use.
2	Let the developer approve, edit, or reject nodes; generate the graph automatically from a prompt plus the real codebase.	The core review step — on its own, this is already a usable first version.
3	Add drift detection — check linked code against its node every time a file is saved or committed.	The one thing no other tool currently does — this is the real differentiator.
4	Add the automatic fast/structured decision, so small changes skip the full review.	That the tool stays fast and doesn't annoy people on small edits.
5	Automatically send approved tasks to the installed coding agent, instead of the developer copying them over by hand.	The full loop working end to end, with no manual steps left.

11 How We'll Know It's Working

The right way to judge this is by whether people keep coming back to it, not by how flashy the launch is:

- **Main signal:** does a developer open the graph again on a second and third feature, without being reminded to? DeepWiki's own numbers are a useful bar to compare against — around 15 minutes per visit on average, and most people coming back directly rather than arriving fresh from a search.
- **How often plans get approved as-is.** If most nodes are approved without edits, the planning step is well-grounded. If people are constantly rewriting the generated plan, the codebase-reading step needs work.
- **How often drift actually gets caught.** Every real catch — a case where the tool flags code that quietly stopped matching its plan — is direct proof the core idea works.
- **How often the fast/structured decision is right.** Tracked by how often a developer manually overrides which path was chosen — frequent overrides mean the threshold needs adjusting.

12 Risks — What Could Go Wrong

- **The fast path picks wrong, silently.** If a risky change is mistakenly treated as trivial, that's worse than having no tool at all, because nobody double-checks it. The rule has to be: when unsure, always play it safe and ask for review.
- **People get tired of reviewing.** Even a nice graph feels like a chore if it shows up too often for small things. Getting the fast/structured decision right is genuinely the hardest part of this whole product — harder than the graph itself — and it should be tuned using real usage, not guessed at upfront.
- **The plan itself could go stale.** If drift detection (Step 3 above) doesn't get built properly, PlanMap turns into exactly the same "write it once and forget it" problem that markdown-based tools already have.
- **Quality depends on tools PlanMap doesn't control.** The actual code is still written by Claude Code, Copilot, or Cursor — PlanMap controls what they're shown and when, not how good their output is.
- **The core idea hasn't been tested with real people yet.** Everything here rests on the belief that a living graph beats a static document. Step 0 exists specifically to find that out cheaply, before any real building starts.

13 What We're Not Building Right Now

Keeping this list short on purpose — it protects the timeline for a solo builder:

- Building a whole new code editor from scratch — this ships as an add-on to editors people already use.
- Building a custom AI model to write code — existing tools already do this well.
- Building custom logic to pick between different AI models — free, ready-made tools already handle this if it's ever needed.
- Extra lenses beyond Business and Security — worth adding later, not needed to prove the idea.
- Support for teams reviewing plans together — the first version is for one developer working alone.

This document reflects the idea as it's been shaped through discussion so far, and should be treated as a living draft — fittingly, the same discipline this product is built around (keep the plan honest, and update it when reality changes) is the discipline this document should be held to as well.